



fx public.purchase

Documentation Dell DVD Store

Schema public

Name purchase

Function to purchase a DVD film.

▼ Input/Output

Mode		Name	Data type
1	OUT	Returns	integer
2	IN	customerid_in	integer
3	IN	number_items	integer
4	IN	netamount_in	numeric
5	IN	taxamount_in	numeric
6	IN	totalamount_in	numeric
7	IN	prod_id_in0	integer
8	IN	qty_in0	integer
9	IN	prod_id_in1	integer
10	IN	qty_in1	integer
11	IN	prod_id_in2	integer
12	IN	qty_in2	integer
13	IN	prod_id_in3	integer
14	IN	qty_in3	integer
15	IN	prod_id_in4	integer
16	IN	qty_in4	integer
17	IN	prod_id_in5	integer
18	IN	qty_in5	integer
19	IN	prod_id_in6	integer
20	IN	qty_in6	integer
21	IN	prod_id_in7	integer
22	IN	qty_in7	integer



		prod_id_in0	integer
24	IN	qty_in8	integer
25	IN	prod_id_in9	integer
26	IN	qty_in9	integer

▼ Script

```
CREATE OR REPLACE FUNCTION public.purchase(customerid_in integer, number_items integer, netar
    RETURNS integer
    LANGUAGE plpgsql
AS $function$
DECLARE
    date_in TIMESTAMP;
    neworderid INTEGER;
    item_id    INTEGER;
    prodid     INTEGER;
    qty        INTEGER;
    cur_quan   INTEGER;
    new_quan   INTEGER;
    cur_sales  INTEGER;
    new_sales  INTEGER;
BEGIN
    date_in := current_timestamp;
    BEGIN
        INSERT INTO ORDERS
        (
            ORDERDATE, CUSTOMERID, NETAMOUNT, TAX, TOTALAMOUNT
        )
        VALUES
        (
            date_in, customerid_in, netamount_in, taxamount_in, totalamount_in
        )
        RETURNING orderid INTO neworderid;

        -- neworderid := CURRVAL('orders_orderid_seq');

        -- ADD LINE ITEMS TO ORDERLINES

        item_id := 0;

        WHILE (item_id < number_items) LOOP
            prodid := CASE item_id WHEN 0 THEN prod_id_in0
                                WHEN 1 THEN prod_id_in1
                                WHEN 2 THEN prod_id_in2
                                WHEN 3 THEN prod_id_in3
                                WHEN 4 THEN prod_id_in4
                                WHEN 5 THEN prod_id_in5
                                WHEN 6 THEN prod_id_in6
                                WHEN 7 THEN prod_id_in7
                                WHEN 8 THEN prod_id_in8
                                WHEN 9 THEN prod_id_in9
                                END;

            qty := CASE item_id WHEN 0 THEN qty_in0
                                WHEN 1 THEN qty_in1
```



```

                                WHEN 5 THEN qty_in5
                                WHEN 6 THEN qty_in6
                                WHEN 7 THEN qty_in7
                                WHEN 8 THEN qty_in8
                                WHEN 9 THEN qty_in9
                                END;

SELECT QUAN_IN_STOCK, SALES INTO cur_quan, cur_sales FROM INVENTORY WHERE PROD_ID=prodid;
new_quan := cur_quan - qty;
new_sales := cur_sales + qty;

IF (new_quan < 0) THEN
    RAISE EXCEPTION 'Insufficient Quantity for prodid:%' , prodid;
    RETURN 0;
ELSE
    UPDATE INVENTORY SET QUAN_IN_STOCK=new_quan, SALES=new_sales WHERE PROD_ID=prodid;
    INSERT INTO ORDERLINES
        (
            ORDERLINEID, ORDERID, PROD_ID, QUANTITY, ORDERDATE
        )
    VALUES
        (
            item_id + 1, neworderid, prodid, qty, date_in
        );

    INSERT INTO CUST_HIST
        (
            CUSTOMERID, ORDERID, PROD_ID
        )
    VALUES
        (
            customerid_in, neworderid, prodid
        );

    item_id := item_id + 1;
END IF;
END LOOP;
RETURN neworderid;
END;
END;
$function$
```